

Floating Point CUDA Implementation of Electron Beam Lithography Proximity Effect Correction for Higher Quality Structures

George Trupiano, Andrew Radwan, Ruben Parra Montenegro

Electrical and Computer Engineering Department

School of Engineering and Computer Science

Oakland University

Rochester, MI

gtrupiano@oakland.edu, radwan@oakland.edu, parramontenegro@oakland.edu

Abstract—Electron Beam Lithography is the process of creating nanoscale structures using a focused beam of electrons. Due to its use in semiconductor manufacturing and other cutting edge fields, demand has only increased over time and will continue to do so. A persisting problem with the technology is the scattering of electrons from the beam. At the nanoscale, electrons do not act like rays, instead they scatter upon contact with the resist and substrate. Any attempt to expose a single point will expose the point and to a lesser degree the area surrounding the point as well. This fact must be taken into account when designing lithographic mask layouts. Previous research has been conducted in parallelizing this operation by altering what parts of the substrate are exposed. This was accomplished by iteratively changing the mask based on whether a location was exposed or not exposed. This paper proposes an improvement by not only altering the locations the beam is applied but also the amount of time the beam dwells at each location. Locations can have any level of exposure instead of the binary exposed and not exposed with the goal of the entire substrate being perfectly exposed. This method provides 3-5x better MSE than a binary implementation at the cost of a 2x slowdown scaling with input size.

Index Terms—Proximity effect correction (PEC), Electron beam lithography (EBL), Integrated circuits (IC), Compute Unified Device Architecture (CUDA)

I. INTRODUCTION

Electron beam lithography (EBL) is a maskless fabrication technique that utilizes a focused beam of electrons to write patterns directly into resist that is sensitive to electrons. The beams can be focused to a spot only a few nanometers wide, allowing EBL to achieve feature sizes below the diffraction limits of optical lithography. Some of the uses of the EBL are photomask fabrication, low-volume prototyping, and research applications. The patterns are written in a sequential manner by a beam that "dwells" at each location of the pattern long enough for the resist to be exposed.

The proximity effect is popular issue in EBL. Electrons do not just deposit their energy at the point of incidence; they forward scatter within the resist and backscatter from the substrate, which spreads energy into areas that are not intended to be exposed. As a result dense regions can accumulate background doses and become overexposed. The same effect

also allows for isolated features to turn out underexposed as the electrons scatter in other directions. The deposited energy can be mathematically modeled as a convolution of the applied dose distribution with a point spread function (PSF), which is commonly modeled as a sum of two Gaussians that represent forward and backward scattering.

Proximity effect correction (PEC) circumvents the issue of blurring by modifying either the geometry of the pattern of the dose that will be applied. Iterative convolution-based PEC algorithms is attractive due to how they use physical PSF models directly, although they are computationally expensive. Each iteration involves a full two-dimensional convolution over the layout. This operation is highly data parallel; every output pixel is computed independently. This means that this algorithm is excellent for running on a GPU.

Prior work by Vang et al. [6] demonstrated a CUDA-parallelized PEC algorithm in which the exposure decision at each location was made in binary, points were either exposed or not. The exposed set was iteratively adjusted until the simulated result closely matched the target layout. This paper is an improvement on this approach; instead of utilizing binary exposure locations, we optimize a floating-point exposure pattern. Locations are exposed to non-negative amounts of time up to a maximum. This addition allows the optimization loop to compensate for the proximity blur with a fine-grained dose adjustment; this cost is mostly in the floating-point operation, but this is all the more reason to use GPU acceleration.

The benefits of using more accurate exposure patterns become more clear as EBL matures into the future as creating sub one nanometer structures will require optimizations in all parts of the process. Previous major advancements in reducing electron spread came though higher energy beams [1] however, increasing voltage past 200 keV can damage the structures it is supposed to be making. The reduction in fuzziness from hyper optimized exposure pattern generation can help work towards the sub one nanometer goal.

The contributions of this work are:

- A continuous (grayscale) dwell-time formulation of iterative PEC that replaces the binary exposed/not-exposed model of prior work.
- An error-driven update rule in which the exposure error is convolved with the PSF, such that each dwell-time pixel is corrected in proportion to its contribution to the surrounding error.
- A CUDA implementation whose image-sized optimization data remain in device memory, built from three kernels: a tiled shared-memory convolution, a parallel squared-error reduction, and an element-wise dwell-time update.
- An evaluation against a sequential reference implementation and a binary implementation, measuring both convergence quality (mean squared exposure error) and runtime.

II. BACKGROUND AND RELATED WORK

A. Electron Scattering and the Proximity Effect

As electrons are focused into a resist, the electrons scatter through two mechanisms; these are illustrated in Figure 1. Forward scattering occurs as the beam travels through the resist, essentially slightly broadening the beam diameter. As the electrons reach the substrate, some of them will undergo collisions and return into the resist as backscattered electrons; those electrons then deposit energy in areas besides where the laser was directed.

The result of these mechanisms is the proximity effect. Dense regions accumulate background dose from their neighbors and are overexposed. Whereas an isolated region would be under dosed due to scattering. Every dose at every location depends on the entire surrounding pattern, so a global adjustment would not be suitable here, and every location must be corrected with respect to every other location, which is what makes these computations demanding.

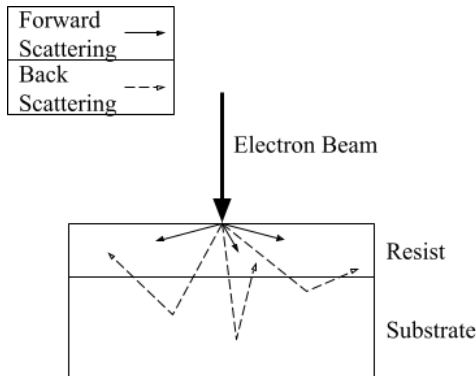


Fig. 1. Forward and Back Scattering in Electron Beam Lithography

B. Proximity Effect Correction Techniques

Proximity effect correction techniques can be broadly categorized into three groups: geometry modulation, which is when pattern edges are pre-distorted so that the developed results match the intended figure; dose modulation, which is when each shape or pixel is assigned a correlated dose; and a technique that combines both. Yao et al. [4] propose a fast and accurate correction that is based on pattern edge shape adjustment. Mao et al. [5] on the other hand, improved the scattering model itself utilizing composite function representation of the electron energy distribution, concluding that correction quality is sensitive to the fidelity of the PSF. The computations in having to solve the system when using model-based dose correction are impractical for large layouts, which is what motivates iterative approximation schemes such as the one utilized in this work.

C. GPU Acceleration of PEC

Every iteration of convolution-based PEC utilizes element-wise image operations and two-dimensional convolutions, which can be parallelized. Every output pixel is computed independently relative to other pixels. This program is run in parallel on the GPU, where every pixel is computed in parallel by thousands of threads. Vang et al. [6] use CUDA to parallelize the iterative PEC algorithm; they represent the substrate as a binary exposure map and then iteratively flip locations between exposed and not exposed based on the simulated result. The work demonstrates substantial speedups over the sequential algorithm; the limiting factor with it is that it constrains every written location to a fixed dose due to the binary exposure map. The binary decision reduces the resolution of the details they are able to achieve in the layouts, leaving errors at edges and corners where the blur is usually the worst. The current work retains the overall simulate, check, and correct flow; still utilizing the GPU-first design, the binary map is replaced with a continuous floating-point dwell time map that is updated by convolutional error feedback.

III. OPTIMIZATION LOOP AND TECHNIQUES

The optimization loop is comprised of the following steps as seen in Fig. 2.

- 1) **Deposited Energy:** Convolve the current dwell times with the PSF mask. This will generate an image containing where energy is deposited and how much is deposited at each location.
- 2) **Mean Squared Error:** Subtract the deposited energy from the desired IC layout. The result is an error image containing the difference between the desired exposure (IC Layout) and the actual deposited energy.
- 3) **Convolved Error Image:** Next the error image is convolved with the PSF mask. This operation assigns blame for what locations specifically are over or under exposing the IC.

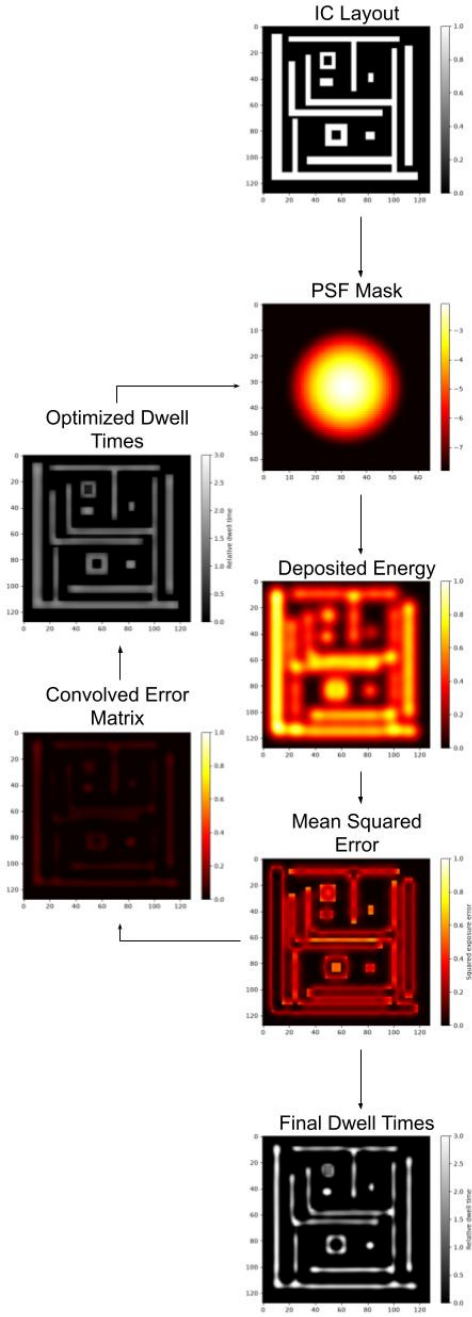


Fig. 2. Flowchart of Optimization Algorithm

- 4) Dwell Time Optimization: Finally, on a pixel by pixel basis, the exposure pattern is adjusted in based on the over or under exposure in the corresponding pixel location in the convolved error image.

A. Energy Deposited by Convolution

The Proximity Effect Correlation can be modeled by a double gaussian function [5] in one dimension as seen in Fig. 3. The equation follows:

$$f(r) = \frac{1}{\pi(1+\eta)} \left[\frac{1}{\alpha^2} \exp\left(-\frac{r^2}{\alpha^2}\right) + \frac{\eta}{\beta^2} \exp\left(-\frac{r^2}{\beta^2}\right) \right] \quad (1)$$

This model can be expanded to a 2D Point Scalar Function (PSF) modeling the spread of energy from a single point. To generate the energy deposited on an IC Layout, the PSF is convolved with the Mask. The convolution implementation is a standard 16x16 block size tiled approach.

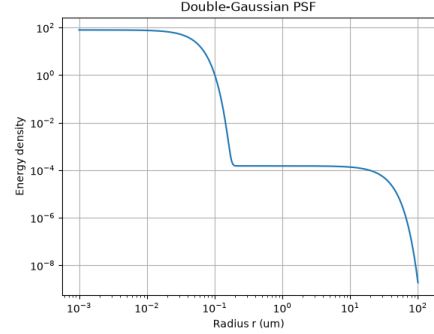


Fig. 3. Double Gaussian Electron Scattering Model

B. Mean Squared Error

Next both the error matrix containing information on where the error is located as well as the mean squared error are calculated. MSE is a scalar used to evaluate whether or not a specific exposure pattern matches an IC layout once convolved. For a 2D image of size (width, height), the mean squared error follows:

$$\text{MSE} = \frac{1}{WH} \sum_{i=1}^W \sum_{j=1}^H [D(i,j) - L(i,j)]^2 \quad (2)$$

First the error matrix must be calculated by subtracting the deposited energy from the desired IC layout. This step is parallelized by assigning one subtraction per thread. To calculate the MSE for the image as a whole, each value from the previous step is squared then saved to shared block level memory. Threads within a block work cooperatively to add the squares together. Finally each block atomically adds into a single value calculating the MSE.

C. Error Convolution

Before adjusting the exposure pattern, the error matrix is convolved with the PSF used earlier. Considering that the energy applied to a pixel not only comes from itself but also its surroundings, this second convolution step assigns blame for where under or over exposure is coming from for each pixel. This step reuses the existing convolution implementation from the first step.

D. Exposure Pattern Adjustment

Finally, the exposure pattern is adjusted to better recreate the IC shape once convolved. Each element of the pattern is adjusted based on its corresponding value in the convolved error image. The value of the error is cubed as a form of simulated annealing such that when the error is large, large changes are made to correct it. The operation is defined as:

$$\text{Updated Dwell Time}(i, j) = \text{Current Dwell Time}(i, j) + (\text{Convolved Error}(i, j)^3 \times \text{Learning Rate}); \quad (3)$$

Where: $0 \leq \text{Updated Dwell Time} \leq \text{Max Dwell Time}$

This operation is parallelized by assigning one element per thread for execution.

IV. METHODS

All tests were performed on Windows 10 with a i7-7700 four core CPU, a GTX 1660 GPU with 1,408 CUDA cores, 6 Gb of VRAM and 16Gb of RAM. Four different optimization programs were tested:

- 1) CUDA C accelerated floating point implementation
- 2) CUDA C accelerated Binary implementation [6] written by C. Vang, O. Yee, N. Kulkarni, and S. Alawneh
- 3) C++ sequential floating point implementation, logically and mathematically identical to the CUDA C version
- 4) A Python implementation of the floating point algorithm used to develop and validate the continuous dwell-time correction algorithm prior to CUDA parallelization.

For result consistency, multiple trials were averaged for each permutation. The CUDA C accelerated Binary implementation was altered to calculate MSE including partial exposure using floating point values, not just one and zero representing exposed and not exposed, respectively. That program was also altered to use libwb logging for output.

V. RESULTS AND DISCUSSION

TABLE I
FLOATING POINT VS SEQUENTIAL EXECUTION TIME

Dataset	Resolution (pixels)	Execution Time (ms)					Float to Python
		Floating Point	Binary	C++	Python		
IC 0	128 × 128	267.8	288.8	11,576.1	3,830.8311	14.3×	
IC 1	256 × 256	458.5	310.4	53,491.7	15,661.5643	34.2×	
IC 2	512 × 512	1,153.8	590.1	221,460.9	59,160.3694	51.3×	
IC 3	128 × 128	280.0	262.9	11,947.2	3,387.9547	12.1×	
IC 4	256 × 256	464.3	300.2	50,863.2	14,537.1875	31.3×	
IC 5	512 × 512	1,135.8	541.1	210,638.4	59,942.7259	52.8×	
IC 6	128 × 128	265.3	248.4	10,846.0	3,818.4288	14.4×	
IC 7	256 × 256	454.2	303.6	49,068.7	15,418.5721	33.9×	
IC 8	512 × 512	1,125.6	556.5	207,308.7	58,682.5059	52.1×	

Table I details the average execution time across ten trials of all three implementations. Compared to a single core, the floating point CUDA implementation performs 14x faster for the smaller 128 size images and up to 52x faster for the 512

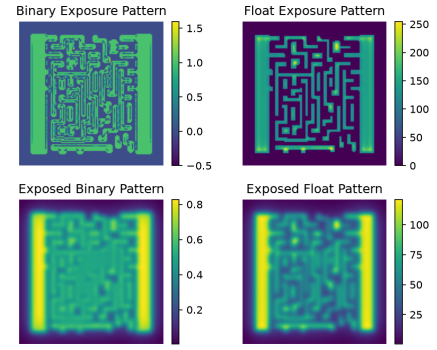


Fig. 4. Comparison of Binary and Floating Point EBL Structures in IC 6

size images. The increase in the time disparity with larger sizes is to be expected considering that the time complexity for square convolution is $O(N^2M^2)$ meaning that as the image size increase the time to execute increases at power-law progression. Comparing the two CUDA approaches, at small image sizes the two operate at about the same execution time. However, as image sizes increase, the binary implementation gains a 2x lead. This lead is due to multiple factors one being the lack of an extra convolution step in calculating the convolved error. Additionally, the solution space for a floating point implementation is naturally vastly larger than that of the binary, this led to the max iteration count set to 100 instead of 64. In testing an iteration count of 1000 was tried with improvements in MSE around .001, well past diminishing returns.

TABLE II
FLOATING POINT TO BINARY MSE WITH MULTIPLICATIVE INCREASE

Dataset	Resolution (pixels)	Final MSE		
		Floating Point	Binary	Float to Binary Increase
IC 0	128 × 128	0.063711	0.289094	4.54×
IC 1	256 × 256	0.054812	0.308273	5.62×
IC 2	512 × 512	0.051542	0.274517	5.33×
IC 3	128 × 128	0.076721	0.289954	3.78×
IC 4	256 × 256	0.072239	0.347875	4.82×
IC 5	512 × 512	0.061978	0.328938	5.31×
IC 6	128 × 128	0.108890	0.331472	3.04×
IC 7	256 × 256	0.109603	0.306051	2.79×
IC 8	512 × 512	0.087659	0.303314	3.46×

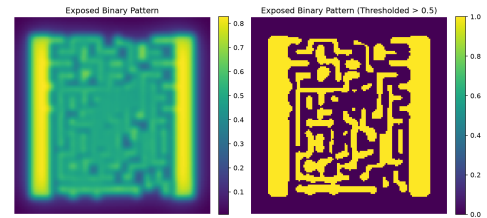


Fig. 5. Effect of Thresholding on image detail in IC 6

Table II details the final MSE of each program, only one floating point program is listed as they have identical output. The floating point method typically achieves a 3-5x better MSE than its binary counterpart. Adjusting dwell time at each location allows for much greater flexibility and more accurate exposure overall. Figure 4 is a comparison between the two algorithms running on the challenging IC six. The top row shows the final exposure pattern generated by the programs whereas the bottom row is the pattern convolved with the PSF to simulate the resist if it was produced. The spacing between lines gets blurry in the binary implementation however, the floating point implementation manages to keep the structure of the IC much more clearly defined. The reason the binary implementation fails to keep spacing is the thresholding approach. After convolution, the binary implementation sets each pixel to one if it is above a set threshold and zero if below. The same blurriness is visible between the gaps of Fig.5. however, after thresholding, the structures are once again well defined. The problem is that the exposure values of the gaps sit between .4 and .5, just below the threshold value of .5. As a result the gaps are nearly as exposed as the structures themselves, but this is ignored by the threshold method.

VI. CONCLUSION

In this work, a CUDA accelerated proximity effect correction algorithm for electron beam lithography was presented. The algorithm replaces the binary exposed model from prior work with a continuous floating point dwell time map. Refining the exposure pattern requires convolving the current dwell times with the PSF, then computing the exposure error against the target IC layout, and finally feeding the PSF convolved error back into a per pixel dwell time update. The image data remains within the device memory through iterations. The loop is built using three kernels: a tiled shared memory convolution, parallel squared error reduction, and an element wise update.

Using nine datasets that span three images sizes, the floating point CUDA implementation ran 14x faster than the sequential python reference at 128×128 and up to about 51x faster at 512×512 , while producing 3-5x better exposure quality compared to the binary implementation. The binary approach does retain a 2x runtime advantage, but the thresholding steps it uses discards near threshold details and blurs the fine spacing between features. The continuous approach preserves the well defined structure boundaries. The results tell us that fine grained dwell time modulation is a worthwhile tradeoff of additional GPU compute for higher pattern fidelity.

REFERENCES

- [1] V. R. Manfrinato, L. Zhang, D. Su, H. Duan, R. Hobbs, E. Stach, and K. K. Berggren, "Resolution limits of electron-beam lithography toward the atomic scale," *Nano Letters*, vol. 13, no. 4, pp. 1555–1558, Apr. 2013, doi: 10.1021/nl304715p.
- [2] V. R. Manfrinato, J. Wen, L. Zhang, Y. Yang, R. G. Hobbs, B. Baker, D. Su, D. Zakharov, N. J. Zaluzec, D. J. Miller, E. A. Stach, and K. K. Berggren, "Determining the resolution limits of electron-beam lithography: direct measurement of the point-spread function," *Nano Letters*, vol. 14, no. 8, pp. 4406–4412, Aug. 2014, doi: 10.1021/nl5013773.
- [3] M. A. Mohammad, T. Fito, J. Chen, S. Buswell, M. Aktary, S. K. Dew, and M. Stepanova, "The Interdependence of Exposure and Development Conditions when Optimizing Low-Energy EBL for Nano-Scale Resolution," in *Lithography*, M. R. Wang, Ed. London, UK: IntechOpen, 2010, doi: 10.5772/8182.
- [4] W. Yao, H. Xu, H. Zhao, M. Tao, and J. Liu, "Fast and accurate proximity effect correction algorithm based on pattern edge shape adjustment for electron beam lithography," *Microelectronics Journal*, vol. 133, Art. no. 105718, Mar. 2023, doi: 10.1016/j.mejo.2023.105718.
- [5] Q. Mao, J. Zhu, X. Cheng, and Z. Wang, "Proximity effect correction in electron beam lithography using a composite function model of electron scattering energy distribution," *Discover Nano*, vol. 20, no. 1, Art. no. 84, May 2025, doi: 10.1186/s11671-025-04264-0.
- [6] C. Vang, O. Yee, N. Kulkarni, and S. Alawneh, "Proximity effect correction in electron beam lithography using CUDA to increase efficiency," in *Proc. IEEE SoutheastCon 2026*, Huntsville, AL, USA, Mar. 2026, doi: 10.1109/SoutheastCon63549.2026.11476633.